

HAND CONTROL

Barmaq hərəkətləri ilə Mouse/Ekran idarəetməsi

Texniki izahat Kitabçası — Source Kod, Kitabxanalar və Arxitektura

Fayl: hand_control.py

Əsas texnologiyalar: OpenCV, MediaPipe Tasks API, PyAutoGUI, NumPy

Layihənin məqsədi: Kameradan real-time əl/barmaq izləmə vasitəsilə sistem mouse-unu jestlərlə idarə etmək

1. Giriş və Layihənin Məqsədi

Bu sənəd `hand_control.py` skriptinin arxitekturasını, istifadə olunan kitabxanaları və hər bir funksiya/dəyişənin nə üçün mövcud olduğunu izah edir. Skriptin əsas vəzifəsi — noutbuk/PC kamerasından gələn görüntüdə istifadəçinin əlini və barmaqlarını real vaxtda aşkar etmək, sonra müəyyən barmaq jestlərini (yalnız işarə barmağının açıq olması, baş barmaqla digər barmaqların toxunması və s.) tanıyaraq bunları kompüterin mouse hərəkətlərinə (kursor yerdəyişməsi, sol/sağ klik, scroll) çevirməkdir.

Layihə üç əsas mərhələdən ibarətdir:

- Görüntü alma — OpenCV vasitəsilə kameradan kadr-kadr video axını oxunur.
- Əl/barmaq analizi — Google-ın MediaPipe kitabxanası ilə hər kadrda əlin 21 sümük nöqtəsinin (landmark) koordinatları hesablanır.
- Jest → Əməliyyat çevrilməsi — həndəsi qaydalarla (məsafə, mövqe müqayisəsi) barmaqların vəziyyəti təyin olunur və PyAutoGUI vasitəsilə əməliyyat sisteminin mouse-u idarə edilir.

2. İstifadə Olunan Kitabxanalar

Aşağıdakı cədvəldə skriptin əvvəlində import edilən hər bir kitabxananın ümumi funksiyası və məhz bu layihədə niyə istifadə olunduğu göstərilir.

Kitabxana / Modul	Layihədə istifadə məqsədi
os	Standart Python modulu — fayl yollarını (path) qurmaq və <code>hand_landmarker.task</code> modelinin diskdə mövcud olub-olmadığını yoxlamaq üçün istifadə olunur.
cv2 (OpenCV)	Kompüter görməsi kitabxanası. Kameranı açmaq (VideoCapture), hər kadri oxumaq, şəkli çevirmək (flip, renk formatı BGR→RGB), ekranda pəncərə açıb video göstərmək və əl skeletini (xətt/dairə) çəkmək üçün istifadə olunur.
mediapipe	Google-ın maşın öyrənməsi əsaslı kitabxanası. Bu layihədə əsas iş yükünü daşıyır: kameradan gələn şəkildə əlin harada olduğunu tapır və 21 barmaq nöqtəsinin dəqiq koordinatlarını (x, y, z) real vaxtda hesablayır.
mediapipe.tasks.py thon (vision, BaseOptions)	MediaPipe-in yeni "Tasks" API-si — köhnəlmiş "solutions" modulunun əvəzinə gəlib. HandLandmarker modelini yükləmək və video axınında əl aşkarlanması aparmaq üçün istifadə olunur.
urllib.request	Standart Python modulu. <code>hand_landmarker.task</code> model faylı diskdə yoxdursa, onu internetdən (Google-un server-indən) avtomatik endirmək üçün istifadə olunur.

pyautogui	Əməliyyat sistemi ilə qarşılıqlı əlaqə qurmaq üçün kitabxana. Tanınan jestlərə əsasən real mouse kursorunu hərəkət etdirmək (moveTo), klikləmək (click) və səhifəni sürüşdürmək (scroll) üçün istifadə olunur — layihənin "idarəetmə" hissəsi.
numpy	Ədədi hesablama kitabxanası. Barmaq koordinatlarını kamera kadrlarının ölçüsündən (məs. 640×480) real ekranın ölçüsünə (məs. 1920×1080) proporsional çevirmək üçün np.interp() funksiyası istifadə olunur.
time	Standart Python modulu. Video kadrları üçün vaxt damğası (timestamp) yaratmaq, həmçinin ardıcıl kliklərin təsadüfən təkrarlanmasının qarşısını almaq (cooldown) üçün istifadə olunur.
math	Standart Python modulu. İki barmaq ucu arasındakı piksel məsafəsini hesablamaq (math.hypot) üçün istifadə olunur — bu məsafə "pinch" (klik) jestinin əsasını təşkil edir.

3. Konfiqurasiya Dəyişənləri (CONFIG)

Skriptin yuxarı hissəsindəki sabitlər proqramın davranışını tənzimləyir. Dəyərləri dəyişməklə kodun məntiqinə toxunmadan həssaslığı tənzimləmək mümkündür.

Element	Nə iş görür
CAM_INDEX	İstifadə olunacaq kameranın nömrəsi (adətən 0 — sistemdəki ilk/əsas kamera).
FRAME_W, FRAME_H	Kamera pəncərəsinin en/hündürlüyü (piksel).
FRAME_MARGIN	Kadrın kənarında qalan "ölü zona". Bu sayədə istifadəçi barmağını kadrın ən kənarına qədər aparmadan bütün ekranı əhatə edə bilər.
SMOOTHING	Mouse hərəkətini yumşaltma əmsalı. Böyük dəyər — daha yumşaq, lakin bir az gecikməli hərəkət; kiçik dəyər — daha sürətli, lakin "titrəyən" kursor.
CLICK_DISTANCE	Piksel ilə hədd: baş barmaqla digər barmaq ucu arasındakı məsafə bundan az olarsa, "pinch" (sıxma) jesti klik kimi qəbul edilir.
CLICK_COOLDOWN	Saniyələrlə vaxt fasiləsi. Bir klikdən sonra növbəti klikin qeydə alınması üçün gözlənilən minimum müddət (təkrar klikin qarşısını alır).
SCROLL_SENSITIVITY	Scroll (səhifə sürüşdürmə) sürətinin əmsalı.

4. MediaPipe Model Qurulması

MediaPipe-in əl aşkarlama funksiyası işləmək üçün öncədən öyrədilmiş bir model faylına (hand_landmarker.task) ehtiyac duyur. Bu fayl iki alt-modeldən ibarətdir: ovucun kadrda harada olduğunu tapan "palm detection" modeli və tapılan ovuc üzərində 21 nöqtəni dəqiqləşdirən "hand landmark" modeli.

ensure_model() funksiyası:

Skript işə düşən kimi çağırılır. Model faylı layihə qovluğunda mövcud deyilsə, onu Google-un rəsmi server-indən (storage.googleapis.com) endirib diskə yazır. Fayl artıq varsa, təkrar endirmə baş vermir — bu, hər dəfə internetə ehtiyacı aradan qaldırır.

HandLandmarkerOptions parametrləri:

Element	Nə iş görür
base_options	Modelin fayl yolunu göstərir (model_asset_path).
num_hands=1	Eyni anda yalnız bir əlin izlənməsini təyin edir (performans üçün).

min_hand_detection_confidence	Əlin "tapıldı" sayılması üçün minimum etibarlılıq faizi (0.7 = 70%).
min_tracking_confidence	Artıq tapılmış əlin növbəti kadrlarda izlənməsi üçün minimum etibarlılıq həddi.
running_mode=VIDEO	Modelin ardıcıl video kadrları üzərində işləməsini bildirir (hər kadr üçün vaxt damğası tələb olunur).

5. Əsas Funksiyalar

5.1 fingers_up(landmarks, handedness_label)

Bu funksiya əlin 21 nöqtəsinə baxaraq hər bir barmağın açıq (yuxarıda) yoxsa qapalı olduğunu müəyyən edir və nəticəni lüğət (dictionary) formasında qaytarır, məsələn: {'thumb': 0, 'index': 1, 'middle': 0, ...}.

Məntiq: hər barmağın ucu (TIP) və orta oynaq (PIP) arasındakı mövqe müqayisə edilir — əgər ucu oynaqdan yuxarıdadırsa (kiçik y dəyəri), barmaq açıq sayılır. Baş barmaq üçün isə şaquli əvəzinə üfüqi (x oxu) müqayisə edilir, çünki baş barmaq yuxarı deyil, yana doğru açılır. Bundan əlavə, əlin sağ yoxsa sol olması (handedness_label) nəzərə alınır, çünki kameradakı əks (ayna) effektinə görə istiqamət dəyişir.

5.2 lm_to_px(landmark, w, h)

MediaPipe koordinatları 0 ilə 1 arasında normallaşdırılmış formada verir (şəklin ölçüsündən asılı olmayaraq). Bu köməkçi funksiya həmin normallaşdırılmış (x, y) dəyərlərini kadrın faktiki piksel ölçüsünə (w, h) çevirərək real piksel koordinatına (əməli ekran nöqtəsinə) çevirir.

5.3 main() — Əsas Dövr (Loop)

Proqramın "ürəyi". Kamera açılır və sonsuz dövrdə (while True) hər kadr üçün aşağıdakı addımlar təkrarlanır:

- Kadr oxunur və güzgü effekti üçün üfüqi çevrilir (cv2.flip).
- Kadr RGB formatına çevrilib MediaPipe-ə ötürülür, nəticədə əl landmark-ları alınır.
- Tapılmış əl varsa, skeleti (21 nöqtə + xətlər) ekranda göstərmək üçün çəkilir.
- fingers_up() ilə hər barmağın vəziyyəti təyin olunur.
- Yalnız işarə barmağı açıqdırsa → mouse kursoru np.interp() ilə ekran koordinatına çevrilərək hərəkət etdirilir (SMOOTHING ilə yumşaldılmış şəkildə).
- Baş barmaq–işarə barmağı məsafəsi CLICK_DISTANCE-dən azdırsa → sol klik.
- Baş barmaq–orta barmaq məsafəsi CLICK_DISTANCE-dən azdırsa → sağ klik.
- İşarə + orta barmaq ikisi də açıqdırsa, onların şaquli mövqe dəyişikliyinə görə → scroll.
- İstifadəçi ESC düyməsinə basdıqda dövr dayanır, kamera buraxılır (release) və bütün pəncərələr bağlanır.

6. Barmaq Landmark İndeksləri

MediaPipe hər əl üçün 21 nöqtə (landmark) qaytarır, hər biri 0-dan 20-dək nömrələnib. Skriptdə yalnız hərəkət və klik məntiqi üçün lazım olan "tip" (barmaq ucu) və "pip" (orta oynaq) nöqtələri istifadə olunur:

Barmaq	Landmark ID	Layihədə rolu
thumb	4 (tip) / 2 (pip)	Baş barmaq — klik (pinch) jestlərində istinad nöqtəsi kimi istifadə olunur.
index	8 (tip) / 6 (pip)	İşarə barmağı — mouse kursorunun hərəkətini idarə edir.
middle	12 (tip) / 10 (pip)	Orta barmaq — sağ klik və scroll jestlərində istifadə olunur.
ring	16 (tip) / 14 (pip)	Üzük barmağı — scroll jestinin "yalnız iki barmaq" şərtini yoxlamaq üçün.
pinky	20 (tip) / 18 (pip)	Çeçələ barmaq — cari versiyada əlavə jestlər üçün ehtiyat saxlanılıb.

Nümunə kod parçası — mouse hərəkəti hesablanması:

```
x = np.interp(index_px[0], (FRAME_MARGIN, w - FRAME_MARGIN), (0, SCREEN_W))
y = np.interp(index_px[1], (FRAME_MARGIN, h - FRAME_MARGIN), (0, SCREEN_H))

curr_x = prev_mouse_x + (x - prev_mouse_x) / SMOOTHING
curr_y = prev_mouse_y + (y - prev_mouse_y) / SMOOTHING

pyautogui.moveTo(curr_x, curr_y)
```

np.interp() kamera kadrındakı piksel mövqeyini (FRAME_MARGIN ilə w-FRAME_MARGIN arasında) ekranın tam ölçüsünə (0 ilə SCREEN_W arasında) proporsional olaraq çevirir. Sonrakı iki sətir isə "xətti interpolasiya" (linear interpolation) ilə əvvəlki mövqe ilə yeni mövqe arasında tədrici keçid yaradaraq kursorun sərt tullanmasının qarşısını alır.

7. Ümumi Axın Diaqramı (Mətn Formasında)

```
Kamera kadri
|
v
cv2.flip + BGR->RGB
|
v
mediapipe.tasks.HandLandmarker.detect_for_video()
|
v
21 landmark (x, y, z) + handedness (Left/Right)
|
v
fingers_up() -> hansı barmaqlar açıqdır?
|
+--> yalnız index açıqsa      -> pyautogui.moveTo()   (mouse hərəkəti)
+--> thumb-index məsafəsi az  -> pyautogui.click()   (sol klik)
+--> thumb-middle məsafəsi az -> pyautogui.click(right) (sağ klik)
+--> index+middle açıqdır    -> pyautogui.scroll()   (scroll)
```

8. Yekun Qeydlər

- Skript real-time işlədiyi üçün performans kamera keyfiyyəti, işıqlandırma və kompüterin gücündən asılıdır.
- CLICK_DISTANCE, SMOOTHING və SCROLL_SENSITIVITY dəyərləri istifadəçinin öz təcrübəsinə görə tənzimləyə bilər.
- hand_landmarker.task modeli bir dəfə endirildikdən sonra internet olmadan da işləyir.
- ESC düyməsi proqramı təhlükəsiz şəkildə bağlayır (kamera resurslarını buraxaraq).